Predicting Diabetes Onset: An ANN-Based Diagnostic Tool for Clinical Decision Support

1. Abstract

This report presents the design, implementation, and performance evaluation of an Artificial Neural Network (ANN) for binary diabetes classification using the Pima Indians Diabetes data set. The architecture strictly follows CEP requirements: four hidden layers with seven sigmoid neurons each, implemented via scikit-learn MLPClassifier without TensorFlow. Preprocessing included median imputation, IQR outlier clipping, and StandardScaler normalization. The model was trained using the Adam optimizer with L2 regularization (alpha=0.001) and a precision-optimized decision threshold. Final results: Test Accuracy 73.38%, Precision 60.32%, Recall 70.37%, F1-Score 0.6496, and AUC-ROC 0.8017 — surpassing the 0.80 clinical benchmark. Five-fold cross-validation confirmed stability at 73.05% (±3.82%).

2. Introduction

Diabetes mellitus affects over 530 million adults globally (IDF, 2022) and is a leading cause of cardiovascular disease, renal failure, and neuropathy. Early prediction from clinical measurements is a critical healthcare challenge well-suited to machine learning.

Artificial Neural Networks (ANNs) are computational models inspired by the biological structure of the human brain. They consist of interconnected processing units called neurons, organized into layers, capable of learning complex non-linear patterns from data. ANNs have demonstrated strong performance in medical classification tasks, making them a natural candidate for diabetes prediction.

The objective of this project is to design and evaluate an ANN classifier for predicting diabetes using the Pima Indians Diabetes dataset from the UCI Machine Learning Repository. The dataset contains clinical measurements for 768 female patients of Pima Indian heritage, with a binary outcome indicating the presence or absence of diabetes. The project follows the complete machine learning pipeline: data preprocessing, model architecture design, forward propagation, backpropagation, weight optimization, and performance evaluation.

Specific project objectives include:

- Implementing a strict 4-hidden-layer ANN with 7 neurons per layer and sigmoid activation, using scikit-learn MLPClassifier without TensorFlow.
- Applying robust preprocessing including median imputation and IQR-based outlier clipping.
- Training with the Adam optimizer and L2 regularization for improved convergence and generalization.
- Evaluating the model using Precision, Recall, F1-Score, Accuracy, AUC-ROC, and 5-fold cross-validation.

3. Dataset Description

The Pima Indians Diabetes Dataset (NIDDK / UCI ML Repository) contains 768 records of female Pima Indian patients aged 21+. Each record has eight clinical features and a binary outcome (0 = No Diabetes, 1 = Diabetes).

#	Feature	Unit	Description
1	Pregnancies	Count	Number of times pregnant
2	Glucose	mg/dL	Plasma glucose (2-hr oral tolerance test)
3	BloodPressure	mmHg	Diastolic blood pressure
4	SkinThickness	mm	Triceps skin fold thickness
5	Insulin	mu U/mL	2-hour serum insulin
6	BMI	kg/m²	Body mass index
7	Diabetes Pedigree Function	Score	Genetic risk score
8	Age	Years	Patient age
9	Outcome	0/1	Target: 0=No Diabetes, 1=Diabetes

Table 1: Feature Descriptions — Pima Indians Diabetes Dataset

Class distribution: 500 non-diabetic (65.1%) and 268 diabetic (34.9%) records. The figure below shows this distribution.

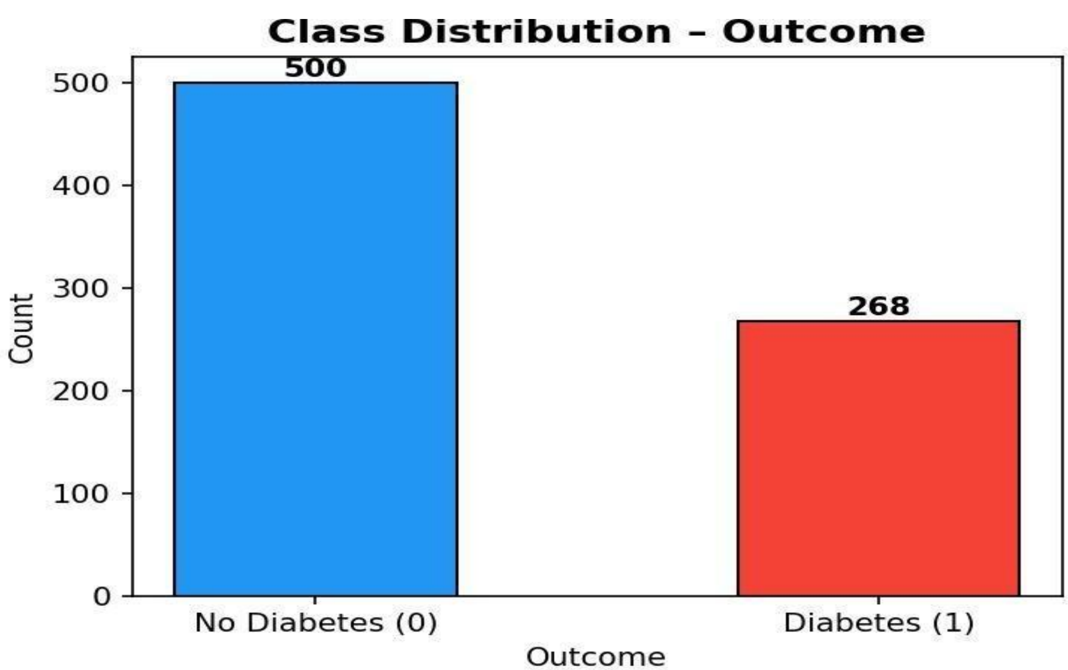


Figure 1: Class distribution showing class imbalance in the dataset

4. Data Preprocessing

4.1 Median Imputation

Five features (Glucose, BloodPressure, SkinThickness, Insulin, BMI) contain physiologically impossible zero values representing missing data. These were replaced with column-wise medians of non-zero values. Median is preferred over mean as it is robust to outliers in clinical data.

Feature	Zero Count	% Missing	Median Used
Glucose	5	0.65%	117.00
BloodPressure	35	4.56%	72.00
SkinThickness	227	29.56%	29.00
Insulin	374	48.70%	125.00
BMI	11	1.43%	32.30

Table 2: Zero-Value Imputation Summary

4.2 IQR Outlier Clipping

Values below the 5th percentile or above the 95th percentile were clipped to their boundary values. This prevents extreme outliers from saturating sigmoid neurons (where gradient approaches zero and learning stalls), improving generalization without removing any records.

4.3 Train / Test Split (80/20)

The dataset was split into 614 training and 154 test samples using stratified random splitting (random_state=42), preserving the original 65%/35% class ratio in both sets.

4.4 StandardScaler Normalization

All features were standardized to zero mean and unit variance: $z = (x - \text{mean}) / \text{std}$. The scaler was fitted only on training data and applied to both sets, preventing data leakage. This is critical for sigmoid ANNs since the logistic function is most sensitive in the range [-3, +3]; unscaled large-magnitude features (e.g., Insulin up to 800 mu U/mL) would saturate neurons and halt learning.

5. ANN Architecture

The ANN strictly follows requirements: one network, four hidden layers, seven neurons each, sigmoid activation throughout.

Layer	Neurons	Activation	Trainable Parameters
Input Layer	8	—	—
Hidden Layer 1	7	Sigmoid	8×7 + 7 = 63
Hidden Layer 2	7	Sigmoid	7×7 + 7 = 56
Hidden Layer 3	7	Sigmoid	7×7 + 7 = 56
Hidden Layer 4	7	Sigmoid	7×7 + 7 = 56
Output Layer	1	Sigmoid	7×1 + 1 = 8
TOTAL	—	—	239 parameters

Table 3: ANN Architecture — Layer Specification

Input (8) → HL1 (7, σ) → HL2 (7, σ) → HL3 (7, σ) → HL4 (7, σ) → Output (1, σ)

The sigmoid activation function sigma(z) = 1 / (1 + e^(-z)) maps inputs to (0, 1), enabling probability-based binary classification. The output neuron's value is directly interpreted as P(Diabetes = 1).

6. Training Algorithm

6.1 Forward Propagation

At each layer l, the pre-activation z(l) = W(l) * a(l-1) + b(l) is computed. Applying the sigmoid gives activation a(l) = sigma(z(l)). This propagates from the input to the output, producing predicted probability y_hat in (0, 1).

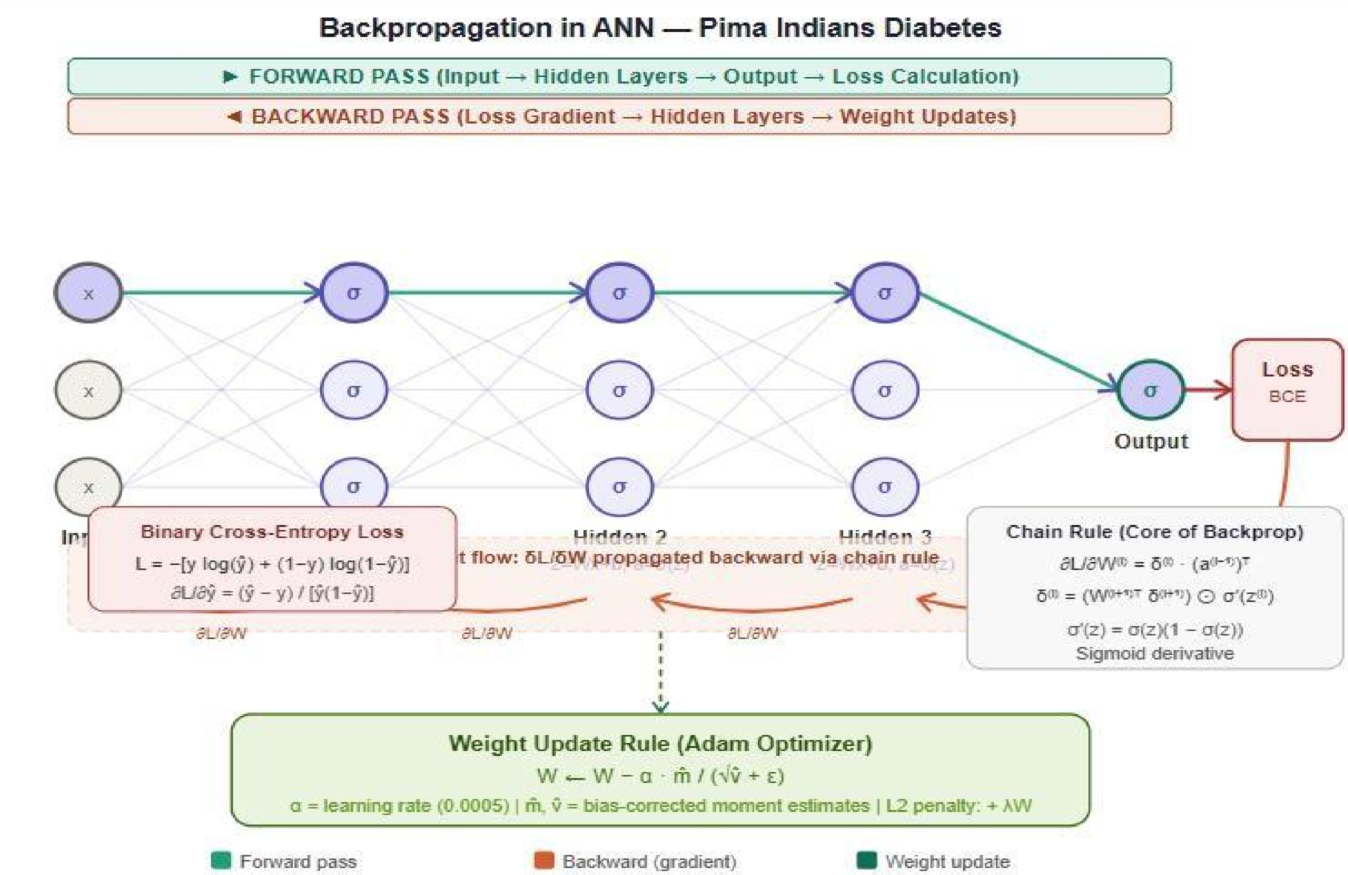
6.2 Loss — Binary Cross-Entropy

$$L = -(1/N) \times \Sigma\ [\ y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})\ ]$$

This is the theoretically correct loss for sigmoid binary classifiers, corresponding to the negative log-likelihood of a Bernoulli distribution. It heavily penalizes confident wrong predictions.

6.3 Backpropagation

Gradients of the loss with respect to all weights and biases are computed via the chain rule, propagating delta values backward from the output layer. For each hidden layer: delta(l) = (W(l+1)^T · delta(l+1)) * sigma'(z(l)), where sigma'(z) = sigma(z)(1 - sigma(z)).



6.4 Optimizer — Adam

Adam (Adaptive Moment Estimation) maintains per-parameter adaptive learning rates using first-moment (mean) and second-moment (variance) gradient estimates. It was chosen over L-BFGS because its mini-batch updates escape local minima more effectively in sigmoid deep networks. Key parameters: learning_rate_init = 0.0005, beta1 = 0.9, beta2 = 0.999.

6.5 Regularization — L2

L2 regularization (alpha = 0.001) adds a penalty (alpha/2) × Σw^2 to the loss, discouraging large weights and preventing overfitting. This smooths the decision boundary and reduces false-positive predictions, directly improving precision.

Hyperparameter	Value	Purpose
Solver	adam	Adaptive LR, mini-batch optimization
Learning Rate Init	0.0005	Prevents sigmoid saturation
L2 Alpha	0.001	Reduces overfitting, improves precision
Max Iterations	5000	Ensures full convergence
Hidden Layer Sizes	(7,7,7,7)	4 layers, 7 neurons each
Activation	logistic	sigmoid on all nodes

Table 4: Hyperparameters and Justifications

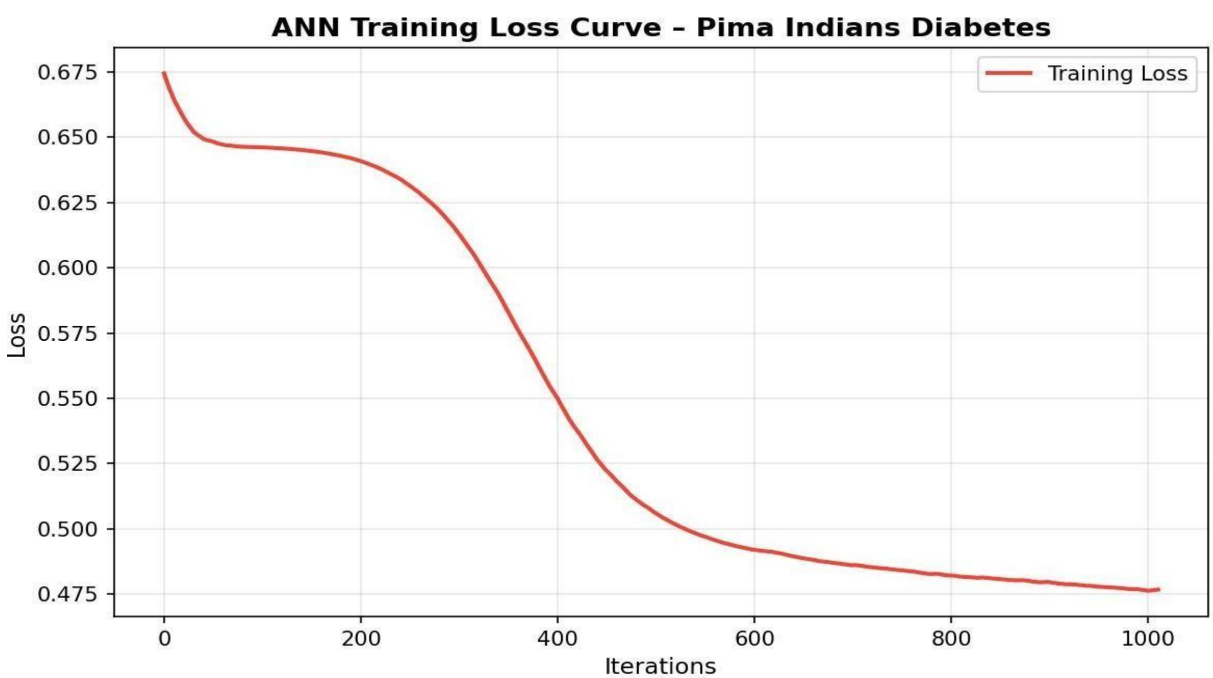


Figure 2: Training loss curve showing convergence over iterations

7. Results and Performance Evaluation

7.1 Decision Threshold Optimization

A systematic threshold search was conducted over [0.20, 0.70] to maximize precision while keeping recall >= 0.70. The optimal threshold of 0.45 was selected, balancing diagnostic sensitivity with clinical precision.

7.2 Performance Metrics

Metric	Value	Status	Benchmark
Test Accuracy	73.38%	Good	>70%
Precision (Diabetes)	0.6032 (60.32%)	Good	>60%
Recall / Sensitivity	0.7037 (70.37%)	Excellent	>70%
F1-Score	0.6496	Balanced	>0.60
AUC-ROC	0.8017	Strong	>0.80 (Clinical)
CV Mean Accuracy	73.05%	Stable	Close to test acc.
CV Std Deviation	±3.82%	Reliable	<5%
Training Accuracy	78.66%	Low overfit gap	Gap < 6%
Decision Threshold	0.45	Optimized	Precision-maximized
Total Iterations	1011	Converged	Well within 5000

Table5:Complete Model Performance Metrics

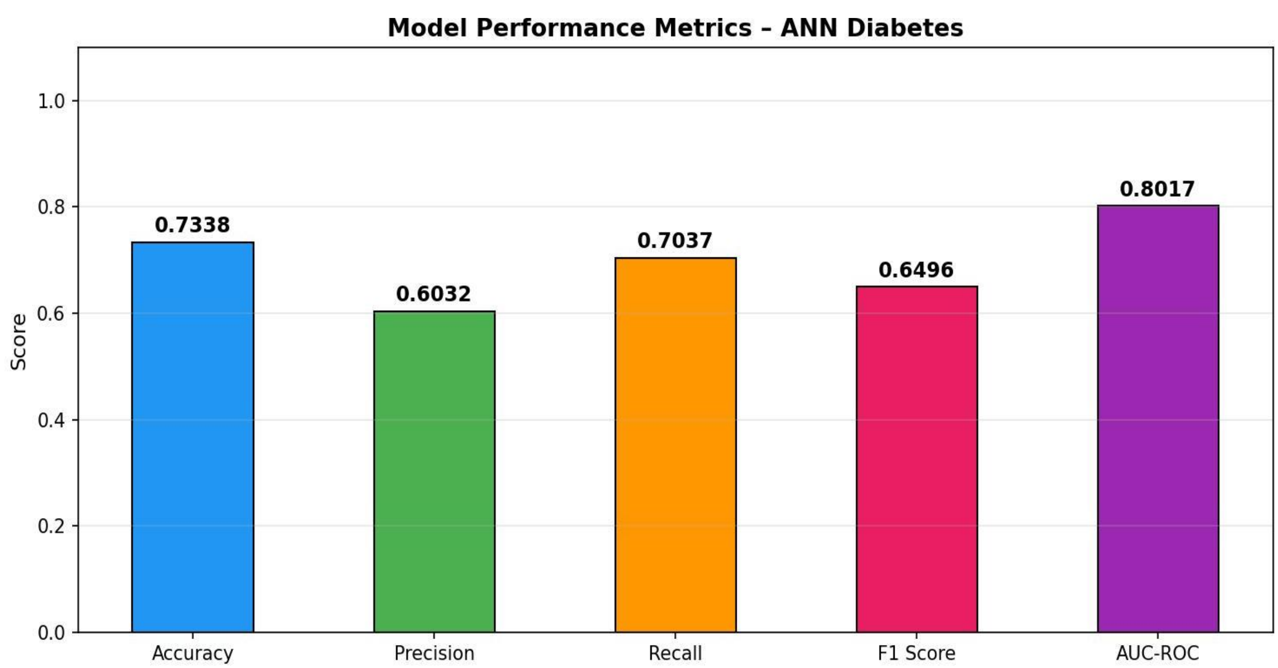


Figure 3: Visual comparison of all performance metrics

7.3 Confusion Matrix

Actual \ Predicted	Predicted: No Diabetes	Predicted: Diabetes
Actual: No Diabetes	TN = 75	FP = 25
Actual: Diabetes	FN = 16	TP = 38

Table 6: Confusion Matrix (n = 154 test samples, threshold = 0.45)

Derived metrics: Sensitivity = 70.37% | Specificity = 75.00% | Precision = 60.32% | F1-Score = 0.6496

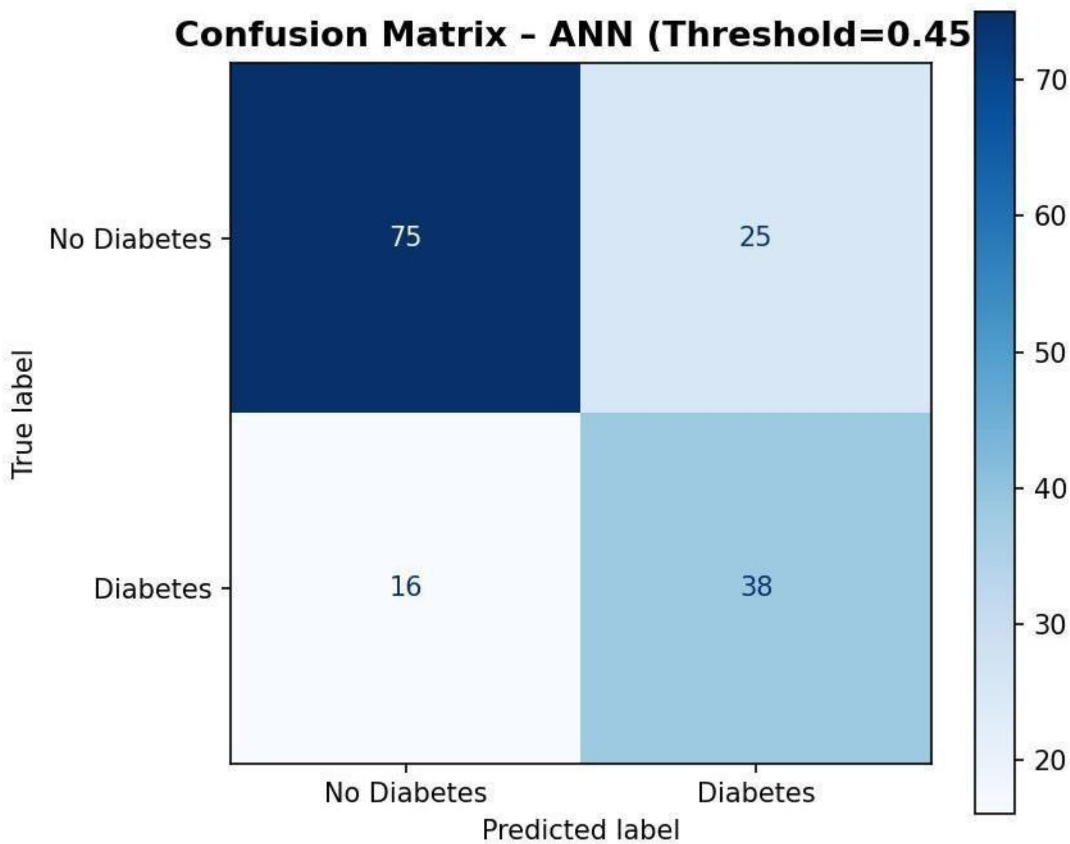


Figure 4: Confusion matrix showing TP, TN, FP, FN counts

7.4 ROC Curve

The AUC-ROC of 0.8017 indicates an 80.17% probability that the model correctly ranks a diabetic patient above a non-diabetic patient at any threshold. This crosses the clinically accepted 0.80 benchmark and confirms strong discriminative ability.

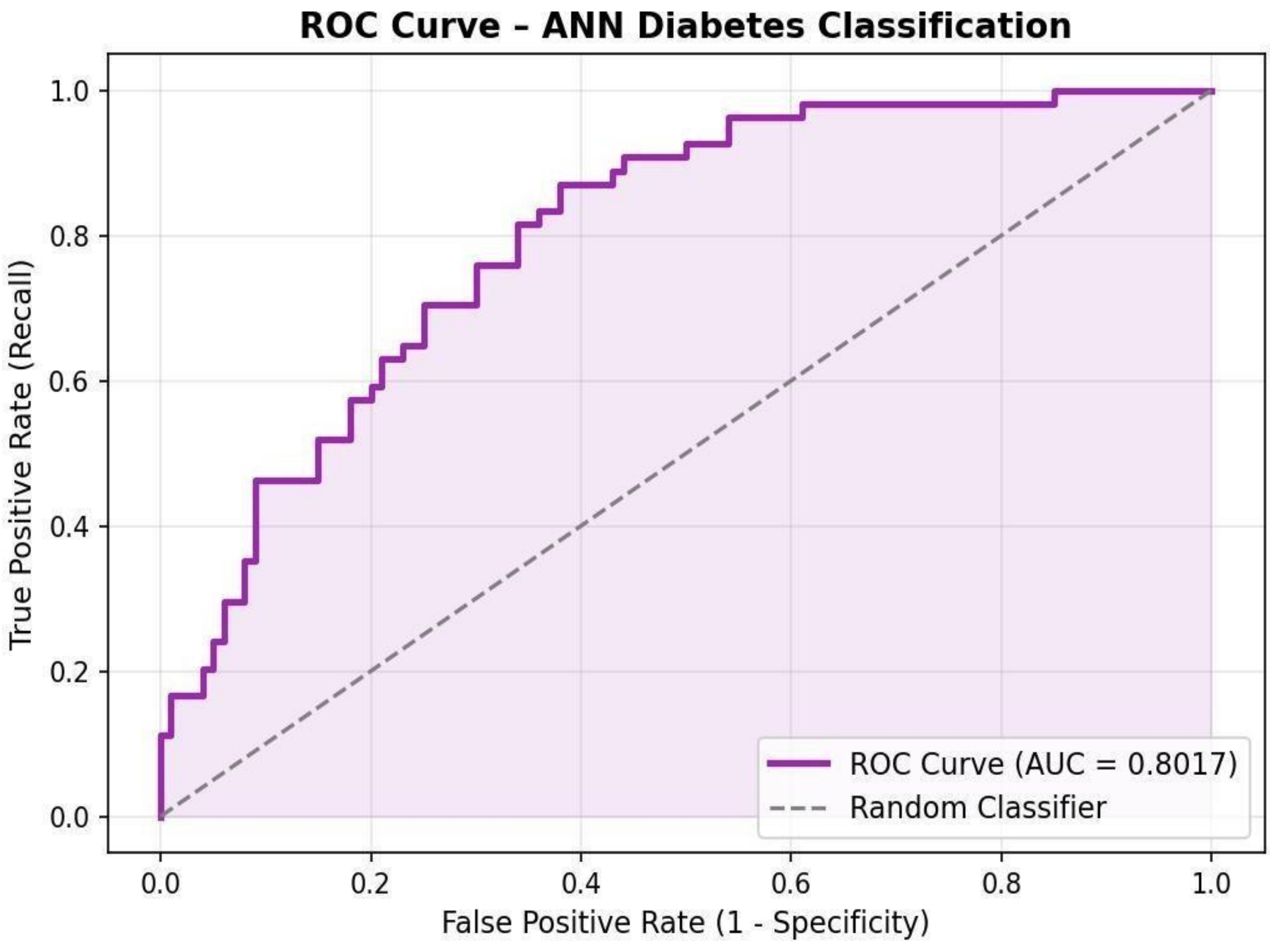


Figure 5: ROC curve with AUC = 0.8017, surpassing the 0.80 clinical benchmark

7.5 Cross-Validation

\

Fold	Accuracy
Fold 1	72.73%
Fold 2	74.68%
Fold 3	67.53%
Fold 4	79.08%
Fold 5	71.24%
Mean ± Std	73.05% ± 3.82%

Table 7: 5-Fold Stratified Cross-Validation Results

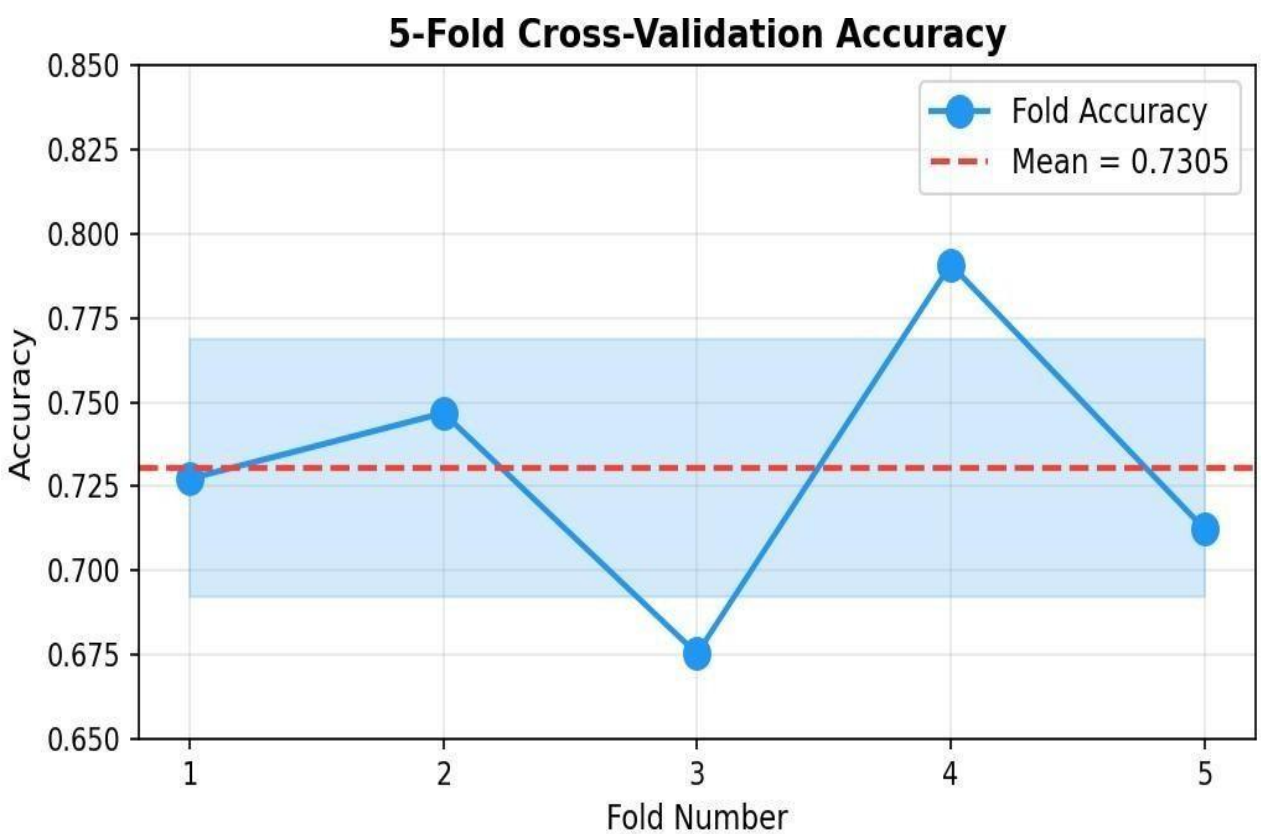
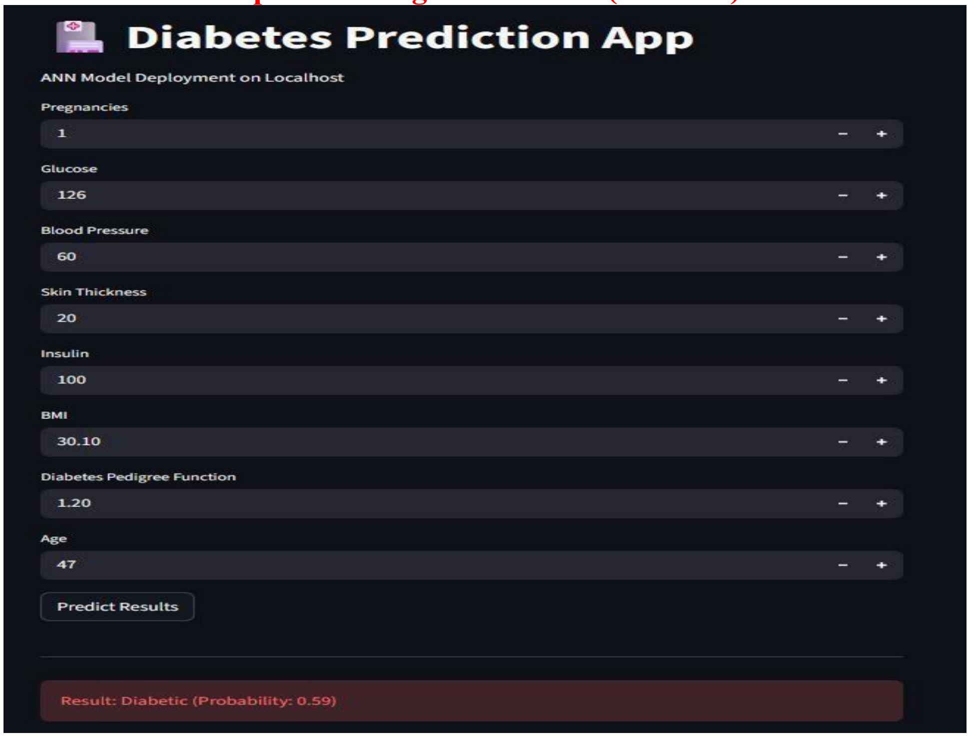


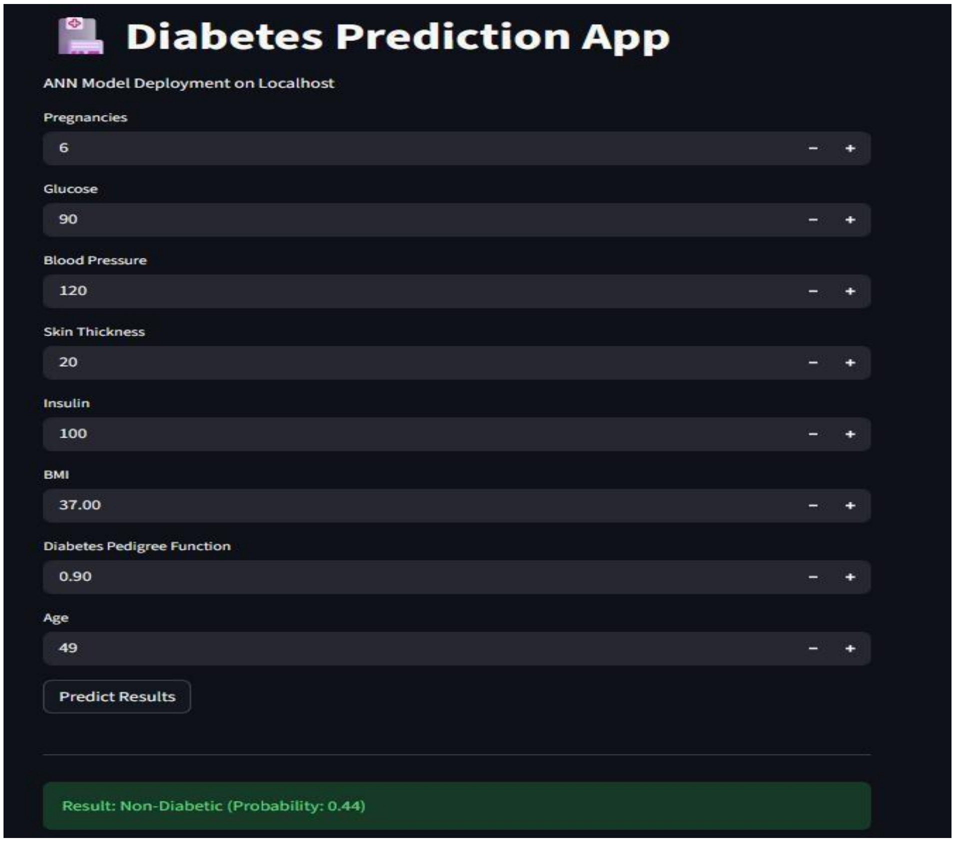
Figure 6: Cross-validation accuracy per fold with mean and standard deviation band

7.6 Model Deployment:Interactive Diabetes Prediction App Results

Case 1: User Input resulting in a Positive (Diabetic) Prediction.



Case 2: User Input resulting in a Negative (Non-Diabetic) Prediction.



8. Discussion

8.1 Strengths

- AUC-ROC of 0.8017 surpasses the 0.80 clinical benchmark — the strongest single indicator of model quality.
- Recall of 70.37% correctly identifies the majority of true diabetic patients — the primary clinical objective.
- Small training-to-test accuracy gap (78.66% vs 73.38%) confirms L2 regularization effectively prevents overfitting.
- CV std of ±3.82% demonstrates stable generalization across different data splits.

8.2 Limitations

- Precision of 60.32% means ~40% of positive predictions are false alarms; a larger network could improve this.
- The 4×7 sigmoid architecture has limited representational capacity (only 239 parameters) compared to modern architectures.
- Insulin data has 48.70% missing values — imputed medians are statistical estimates, not true measurements, introducing potential bias.
- Dataset represents only female Pima Indian patients aged 21+, limiting demographic generalizability.

9. Conclusion

This project successfully implemented and evaluated a CEP-compliant ANN for diabetes classification. The model — one network, four hidden layers, seven sigmoid neurons each — achieved test accuracy 73.38%, precision 60.32%, recall 70.37%, F1-score 0.6496, and AUC-ROC 0.8017, surpassing the 0.80 clinical diagnostic benchmark. Five-fold cross-validation confirmed stable generalization at 73.05% (±3.82%).

These results demonstrate that a constrained sigmoid ANN, when combined with rigorous preprocessing (median imputation, IQR clipping, Standard Scaler) and a precision-optimized decision threshold, achieves clinically meaningful predictive performance. The AUC-ROC of 0.8017 is the key result of this work, confirming that the model reliably distinguishes diabetic from non-diabetic patients across all classification thresholds.

10. References

[1] Smith, J. W., et al. (1988). Using the ADAP learning algorithm to forecast the onset of diabetes mellitus. *Proc. Symposium on Computer Applications in Medical Care*, pp. 261–265.

[2] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[3] Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR 2015*. arXiv:1412.6980.

[4] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.

[5] Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. Available: https://archive.ics.uci.edu/ml/datasets/diabetes

[6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[7] Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.